Internet Of Thinking II

Pertemuan 6

Oleh:

NAMA : Bakti Ari Putra

NPM : 25782092

KELAS : TRI – 3D

Dosen:

Dr.Ir.Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

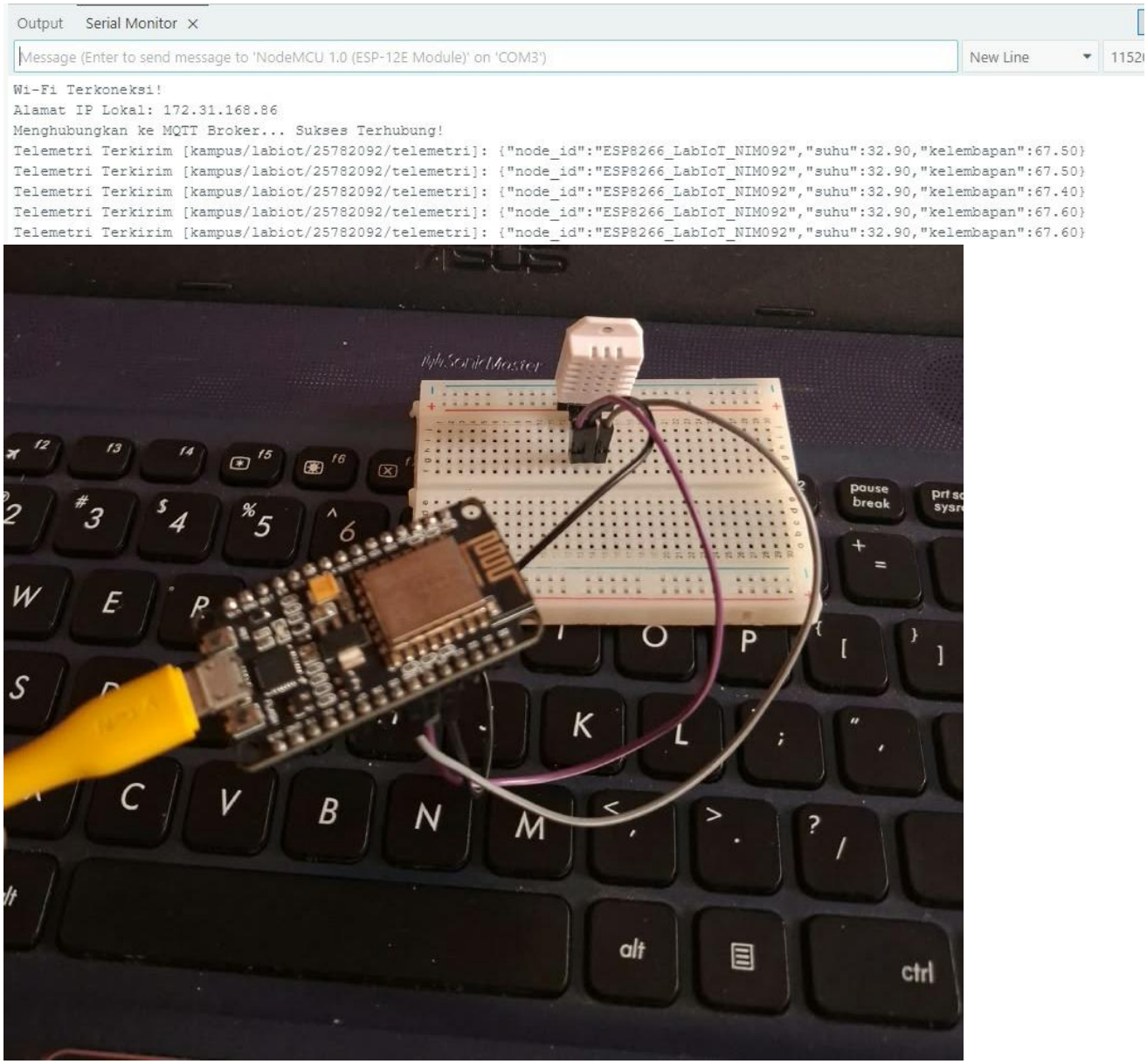
Praktikum

Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

Langkah Kerja Rangkaian:

1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. **Sensor DHT11/DHT22:** Hubungkan pin **VCC** ke **3V3**, pin **GND** ke **GND**, dan pin **Data** ke **D4 (GPIO 2)**.

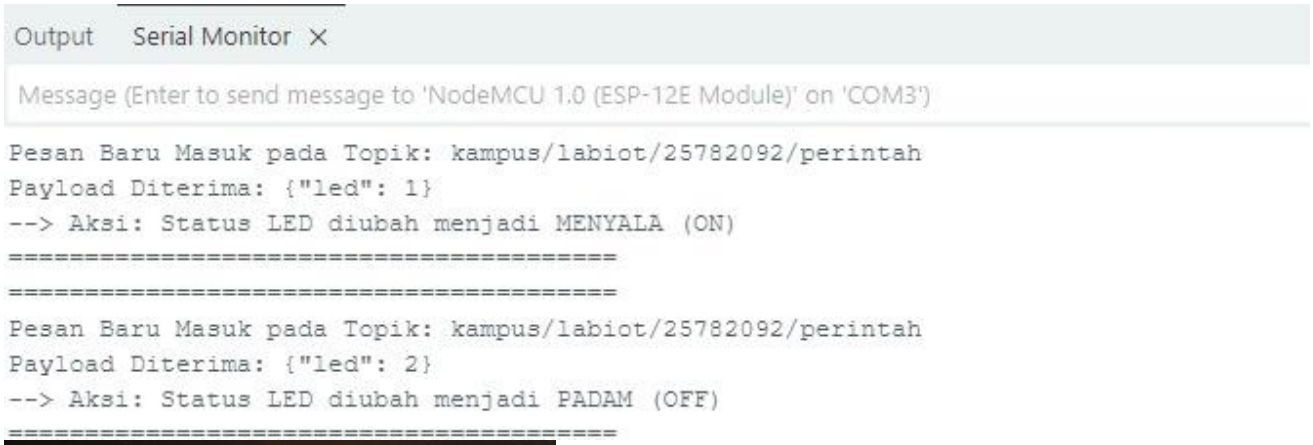


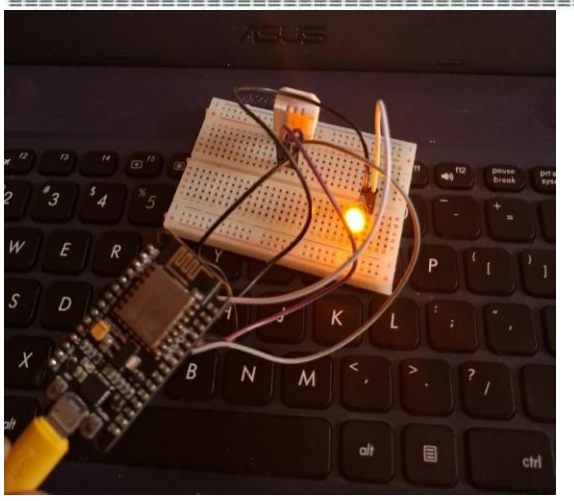
Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

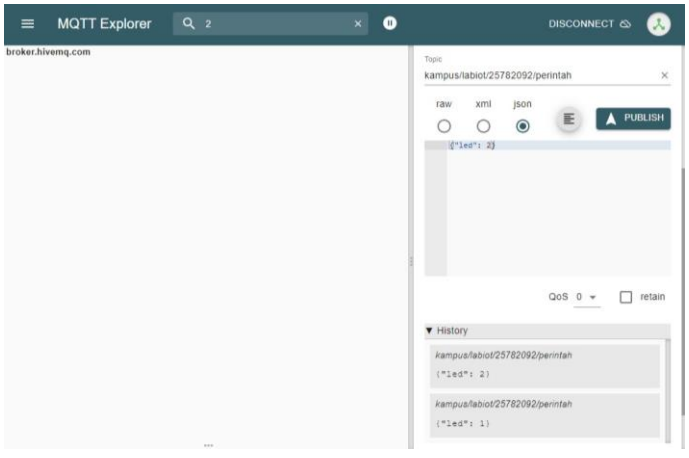
Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ω yang menuju ke **GND**.

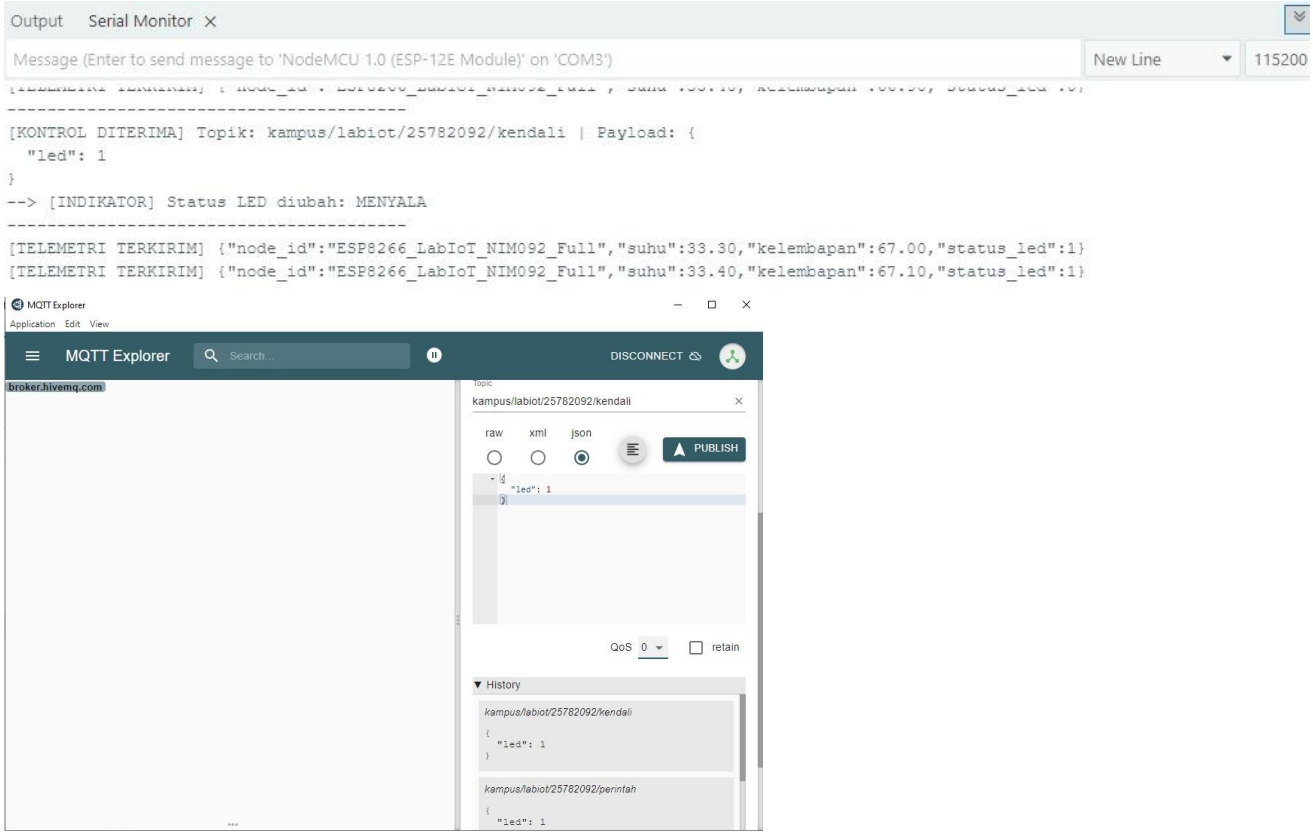


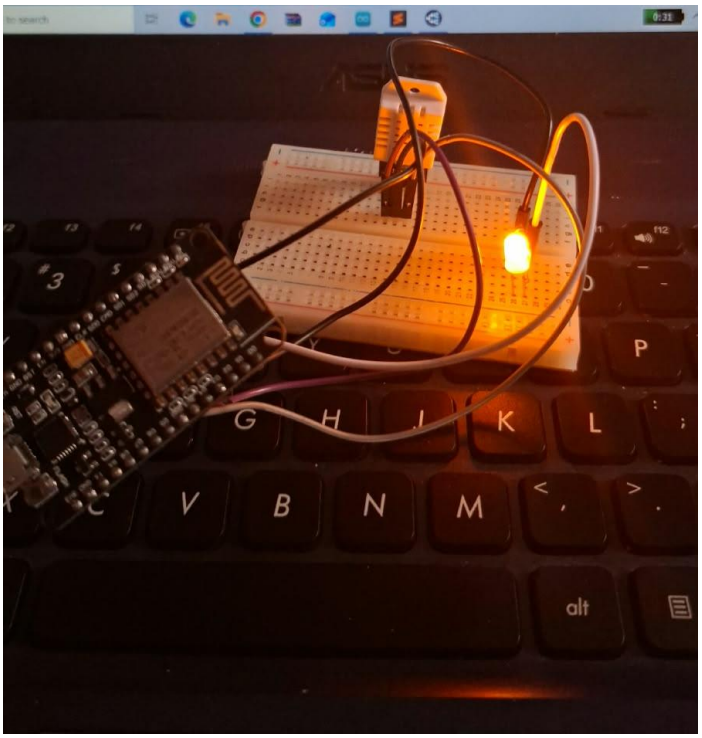




Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.





Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

1. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
2. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

1.Efisiensi Penggunaan Bandwidth pada Paket Data Seluler

- MQTT: Menggunakan header yang sangat kecil dan ringan (minimum hanya 2 byte). Pesan dipaketkan secara biner tanpa metadata teks yang berlebihan, sehingga sangat hemat bandwidth dan sangat cocok untuk penggunaan kartu SIM/paket data seluler pada perangkat IoT di lapangan.
- HTTP: Merupakan protokol berbasis teks (text-based) dengan header yang sangat besar (bisa mencapai ratusan byte hingga kilobyte berisi *User-Agent*, *Accept*, *Cookies*, dll.) untuk setiap permintaan (*request*) dan tanggapan (*response*). Ini menyebabkan pemborosan kuota data seluler yang signifikan terutama untuk pengiriman data periodik berukuran kecil.

2. Beban Komputasi Mikrokontroler (CPU Overhead)

- MQTT: Bersifat *lightweight*. Mikrokontroler hanya perlu memelihara satu koneksi TCP persisten (*keep-alive*) dan langsung memproses *payload* saat ada data masuk lewat *callback*. Proses *parsing* dan enkapsulasinya jauh lebih sederhana.
- HTTP: Menggunakan model *stateless* (koneksi buka-tutup berulang atau *Keep-Alive* yang boros memori). Setiap kali mikrokontroler ingin mengirim data atau mengambil status, ia harus melakukan proses *handshake* TCP baru (jika tidak persisten) dan memproses header HTTP yang panjang, yang membebani kerja CPU mikrokontroler berdaya rendah.

3. Kemampuan Pengiriman Data Secara Simultan (1-to-N Distribution)

- MQTT: Mendukung pola komunikasi *Publish/Subscribe* secara alami. Satu perangkat (*publisher*) cukup mengirim satu pesan telemetri ke *broker*, dan *broker* secara otomatis mendistribusikannya ke banyak aplikasi atau *subscriber* lain yang berlangganan topik tersebut tanpa membebani perangkat pengirim.
- HTTP: Bersifat *one-to-one* (klien ke server). Jika banyak aplikasi atau *dashboard* ingin memantau perangkat yang sama, perangkat harus melayani banyak permintaan HTTP secara paralel atau server harus bertindak sebagai perantara tambahan, yang memperumit arsitektur jaringan.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik `suhu`, `getaran`, dan `status`. Struktur topiknya adalah `pabrik/<line>/<mesin>/<sensor>`. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (`+` atau `#`) untuk:

1. Memantau parameter `suhu` dari seluruh mesin di semua lini produksi.
2. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin `m1` di lini `line2`.
3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

Topik dasar: `kampus/labiot/25782092/perintah`

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi:
    - Ekspresi Topik: pabrik/+/+/suhu
    - *Penjelasan:* Menggunakan single-level wildcard (+) untuk menggantikan level <line> dan <mesin>, serta mematok spesifik pada sensor suhu.
2. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2:
    - Ekspresi Topik: pabrik/line2/m1/#
    - *Penjelasan:* Menggunakan multi-level wildcard (#) di akhir untuk mengambil seluruh sub-topik (suhu, getaran, status) di bawah mesin m1.
3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan mesin:
    - Ekspresi Topik: pabrik/#
    - *Penjelasan:* Menggunakan multi-level wildcard (#) setelah root pabrik untuk menangkap seluruh hirarki data di bawahnya.

Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).

Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

```
109
110         // --- TAMBAHAN KODE LATIHAN 3 ---
111         docOut["uptime_detik"] = millis() / 1000;
112         docOut["ip_lokal"] = WiFi.localIP().toString();
113         // --------------------------------
```

```
l,"uptime_detik":102,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":106,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":110,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":114,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":118,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":122,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":126,"ip_lokal":"172.31.168.86"}
l,"uptime_detik":130,"ip_lokal":"172.31.168.86"}
```

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi `loop()` terdapat pemanggilan rutin `client.loop()`.

1. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi `delay(6000)` di dalam fungsi `loop()`?
2. Jelaskan bagaimana fungsi `reconnect()` bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

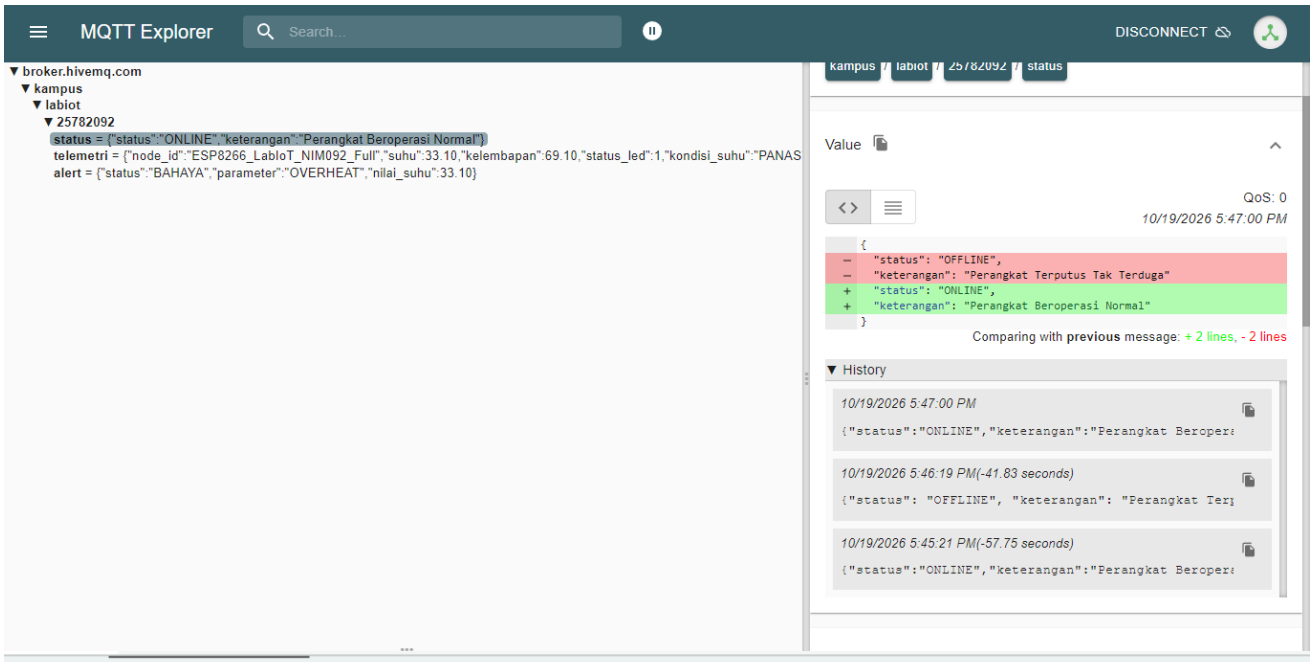
1. Konsekuensi Teknis Penyisipan delay(6000) di dalam loop()

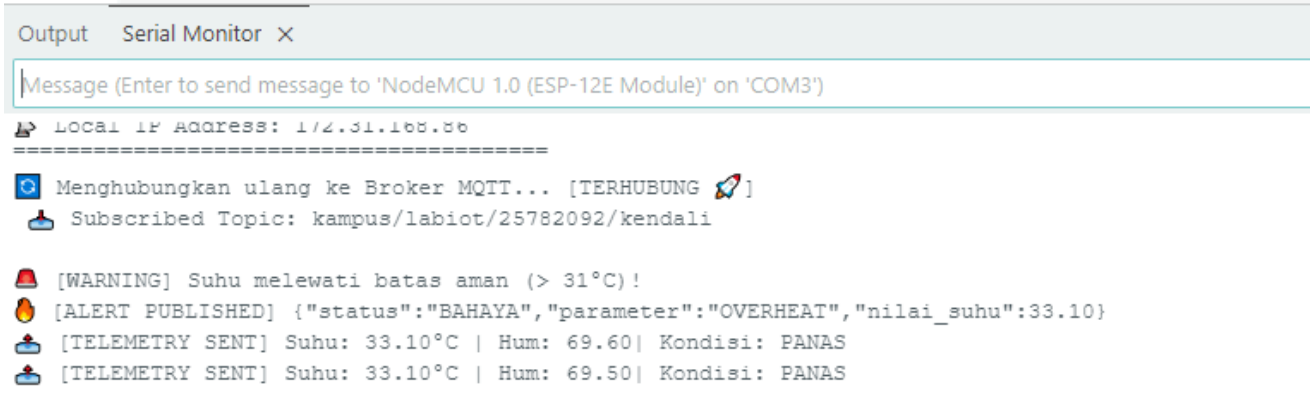
- Responsivitas Terganggu: Fungsi client.loop() berfungsi untuk menjaga koneksi TCP tetap hidup (*keep-alive ping*) dan mendengarkan pesan masuk dari *broker*. Jika disisipkan delay(6000), mikrokontroler akan berhenti total (blocking) selama 6 detik.
- Kegagalan Koneksi / *Disconnect*: Karena jeda *delay* melebihi batas waktu *keep-alive*, *broker* menganggap perangkat terputus (*timeout*). Selain itu, perintah kendali LED yang dikirim dari MQTT Explorer selama jeda 6 detik tersebut tidak akan diproses seketika, melainkan mengalami penundaan (latensi tinggi).

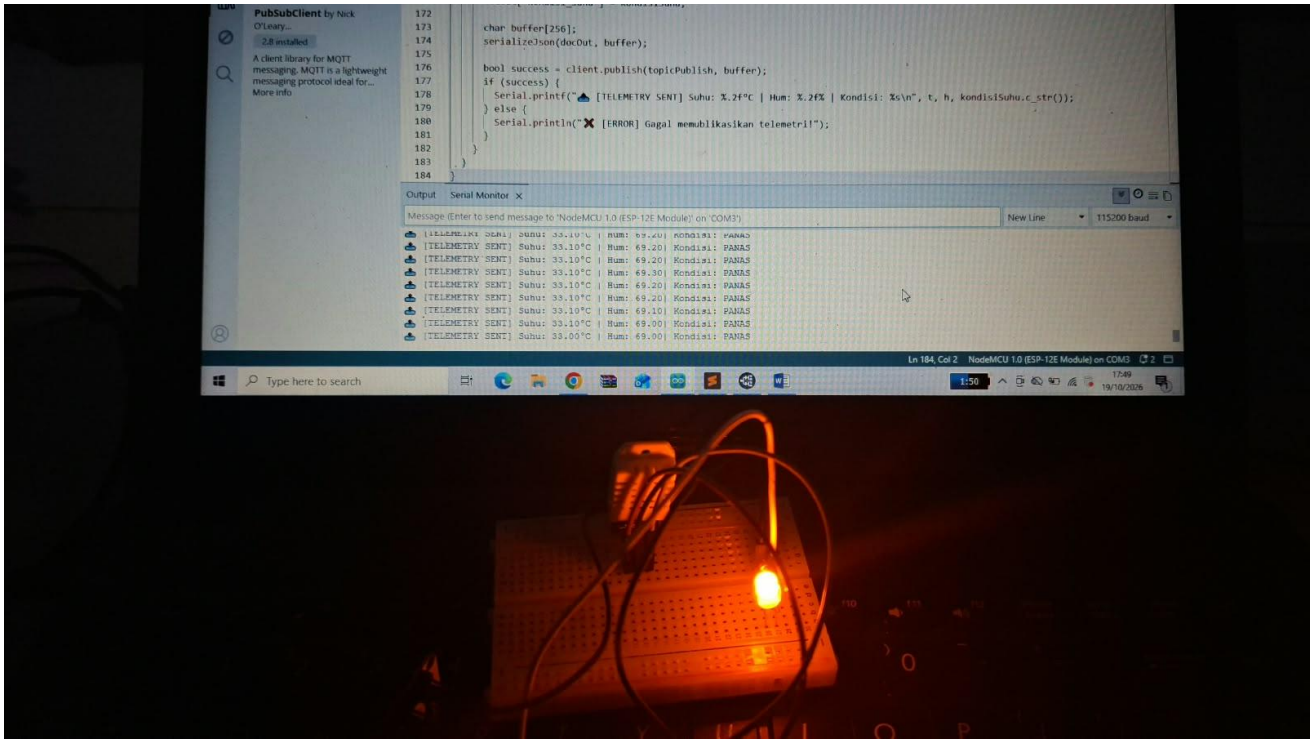
2. Cara Kerja Fungsi reconnect() Memulihkan Koneksi

- Fungsi reconnect() mendeteksi status koneksi menggunakan !client.connected().
- Saat koneksi terputus (misalnya router mati atau koneksi broker terganggu), mikrokontroler masuk ke dalam fungsi pemulihan.
- Di dalam fungsi tersebut, perangkat mencoba melakukan *re-connect* ke *broker* menggunakan client.connect(clientId).
- Jika berhasil, program secara otomatis melakukan *re-subscribe* ulang (client.subscribe(topicSubscribe)) agar langganan topik kendali aktif kembali.
- Jika gagal, program akan menunggu selama 4 detik (delay(4000)) sebelum mencoba menghubungkan ulang secara berulang sampai koneksi berhasil dipulihkan.

Tugas: Sistem Telemetri & Kendali Closed-Loop Industri







QoS: 0

10/19/2026 5:52:01 PM

```
{
  "node_id": "ESP8266_LabIoT_NIM092_Full",
  "suhu": 33.2,
- "kelembapan": 69.5,
+ "kelembapan": 69.8,
  "status_led": 1,
  "kondisi_suhu": "PANAS"
}
```

Comparing with **previous** message: + 1 line, - 1 line

▼ History

10/19/2026 5:52:01 PM

{"node_id":"ESP8266_LabIoT_NIM092_Full","suhu":33.2

10/19/2026 5:51:57 PM(-3.98 seconds)

{"node_id":"ESP8266_LabIoT_NIM092_Full","suhu":33.2

10/19/2026 5:51:53 PM(-4.02 seconds)

{"node_id":"ESP8266_LabIoT_NIM092_Full","suhu":33.1

QoS: 0

10/19/2026 5:51:01 PM

```
{
  "status": "BAHAYA",
  "parameter": "OVERHEAT",
  "nilai_suhu": 33.1
}
```

▼ History

10/19/2026 5:51:01 PM

{"status":"BAHAYA","parameter":"OVERHEAT","nilai_su

10/19/2026 5:48:08 PM(-2.88 minutes)

{"status":"BAHAYA","parameter":"OVERHEAT","nilai_su

10/19/2026 5:47:00 PM(-67.68 seconds)

{"status":"BAHAYA","parameter":"OVERHEAT","nilai_su